

Q1

Convert the following algorithm into a program and find its time complexity using the counter method.

void function (int n)

```
{
    int i= 1;
    int s =1;
    while(s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

SOL:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    int i = 1;
```

```
    int s = 1;
```

```
    int count = 0;
```

```
    scanf("%d", &n);
```

```
    while (s <= n) {
```

```
        count++;
```

```
        i++;
```

```
        count++;
```

```
        s += i;
```

```
        count++;
```

```
    }
```

```
    count++;
```

```
    count++;
```

```
    count++;
```

```
    printf("%d", count);

    return 0;
}
```

SAMP INPUT: 9

O/P: 12

Q2

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("*");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

SOL:

```
#include <stdio.h>
```

```
int main() {
    int n;
    int count = 0;

    scanf("%d", &n);

    count++;
}
```

```

if (n == 1) {
    printf("*");
} else {
    for (int i = 1; i <= n; i++) {
        count++;

        for (int j = 1; j <= n; j++) {
            count++;

            printf("*");
            printf("*");

            count++;
            break;
        }

        count++;
    }

    count++;
}

printf("\n%d", count);

return 0;
}

```

SAMP INPUT: 5

O/P: 22

Q3

Convert the following algorithm into a program and find its time complexity using counter method.

```

Factor(num) {
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
}

```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

SOL:

```
#include <stdio.h>

int main() {
    int num, i;
    int count = 0;

    scanf("%d", &num);

    for (i = 1; i <= num; ++i) {
        count++; // i <= num comparison

        count++; // num % i == 0 comparison
        if (num % i == 0) {
            printf("%d ", i);
        }

        count++; // i++
    }

    count++; // Final failed comparison: i <= num

    printf("\n%d", count);

    return 0;
}
```

SAMP INPUT: 6

O/P: 1 2 3 6
19

Q4

Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

SOL:

```
#include <stdio.h>
```

```
int main() {  
    int n, c = 0;  
  
    scanf("%d", &n);  
  
    for (int i = n / 2; i < n; i++)  
        for (int j = 1; j < n; j = 2 * j)  
            for (int k = 1; k < n; k = k * 2)  
                c++;  
  
    printf("%d", c);  
  
    return 0;  
}
```

SAMP INPUT: 8

O/P: 36

Q5

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)  
{  
    int rev = 0, remainder;  
    while (n != 0)  
    {  
        remainder = n % 10;  
        rev = rev * 10 + remainder;  
        n /= 10;  
    }  
    print(rev);  
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

SOL:

```
#include <stdio.h>

int main() {

    int n, rev = 0, remainder;

    int count = 0;

    scanf("%d", &n);

    while (n != 0) {

        count++;

        remainder = n % 10;

        count++;

        rev = rev * 10 + remainder;

        count++;

        n /= 10;

        count++;

    }

    count++;

    printf("%d", count);

    return 0;

}
```

SAMP INPUT: 1234

O/P: 17